Chapter 4: Introduction to Data and Programming with Python

Extracted Syllabus & Notes on Modules, Scope and NumPy

Class Notes

October 27, 2026

Contents

1 Syllabus Extracted from Textbook Page **2**
- 1.1 Chapter Outline . . . 2
- 1.2 Suggested List of Assignments . . . 2

2 Modules and Packages **3**
- 2.1 Definition . . . 3
- 2.2 Modules vs Libraries . . . 3
- 2.3 Scope and Uses of Packages . . . 3

3 NumPy (Numerical Python) **4**
- 3.1 What is NumPy? . . . 4
- 3.2 Installation . . . 4
- 3.3 Importing NumPy . . . 4
- 3.4 Arrays . . . 4
- 3.5 NumPy Array vs Python List . . . 4
- 3.6 Creating Arrays . . . 5
- 3.7 Mathematical Operators on Arrays . . . 5
- 3.8 Important Array Attributes and Methods . . . 6

4 Brief Note on Pandas (related library) **6**

1 Syllabus Extracted from Textbook Page

1.1 Chapter Outline

1. **Introduction to Data and Programming with Python**

   (i) Modules and Packages

   (ii) Scope and uses of packages, modules and libraries like NumPy, Pandas, SciPy, Matplotlib; implementation using simple programs in Python.

   (iii) An understanding of what libraries are and why we need them in the first place, the syntax of importing libraries.

2. **Lists and Tuples in Python**

   (i) Introduction to lists, creation of list, access elements of a list, list operations (append, insert, extend, sort, search), introduction to tuples, creation of tuple, accessing tuples, deleting a tuple, and converting list to tuple or vice-versa.

3. **Strings**

   (i) Introduction to strings: creating strings, traversing a string, multiple strings, concatenating strings, escape sequences.

   (ii) String operators, string in-built functions: `str.upper()`, `str.lower()`, `str.len()`, `str.find()`, `str.count()`, `str.replace()`, `str.startswith()`, `str.endswith()`, `str.isupper()`, `str.islower()`, `str.isdigit()`, `str.isalpha()`, `str.isalnum()`, `str.title()`, `str.swapcase()`, etc.

4. **Data Exploration**

   (i) Sources of Dataset in AI – Kaggle platform.

   (ii) An understanding of different ways of representing data to gather meaningful information: bar graphs, histograms, line graphs, scatter plots, pie charts.

5. **Modelling and Evaluation**

   (i) Approaches in AI data modelling: Prediction – linear regression, Rule based approach, learning based approach, supervised learning, unsupervised learning (brief understanding of each).

1.2 Suggested List of Assignments

Artificial Intelligence (AI)

1. Make a presentation on the difference between a machine/robot/cobot.
2. Technology is evolving at a fast pace. What does it mean for us? Discuss.
3. Are smart systems making humans less smart? Discuss.
4. Prepare a report on robotics systems being used in different spheres of life.

Python Programming

1. Create a list of all students in your class and sort them in alphabetical order.
2. Find a word string of interest in a given sentence. Display output: "String Name Found" or "String Name Not Found".

3. Make a dictionary dataset of all cities in India and store their average temperature and pollution data. Create NumPy arrays.
4. Create NumPy arrays.
5. Getting elements from NumPy arrays using index values.
6. Getting NumPy array shape and reshaping them.
7. Iterating a NumPy array.
8. Join and split NumPy arrays.
9. Searching and sorting of NumPy arrays.
10. Display an image using matplotlib and print its NumPy array form. Also check the data type and shape.
11. Use the library `storage` for getting sample images.
12. Reading a CSV file into a Pandas DataFrame.
13. Basic data analysis on data with `head()`, `tail()`, `info()` commands.

2 Modules and Packages

2.1 Definition

- A **module** is a Python file containing a collection of Python statements, functions and global variables. It is created with an extension `.py`.
- A **package** is a collection of relevant modules saved under the same directory and given a name.
- There are various packages related to various purposes available for free to be used in Python.

2.2 Modules vs Libraries

- **Modules** are individual files that contain code, functions, or classes for code organisation and reusability. They can be imported into other files using the `import` statement.
- **Libraries** are collections of precompiled code, functions, or resources that provide additional functionality. They solve specific problems or offer specific capabilities and can be installed and imported into Python projects.
- Modules help organise and reuse code within a program, while libraries extend Python's capabilities by providing ready-made solutions for various tasks or domains.

2.3 Scope and Uses of Packages

Since a package organises related modules, sub-packages, and resources into a hierarchical structure, it provides a large set of predefined functions, classes, and methods for various tasks.

The scope of the packages lies in:

Organizing code It organises the related modules into a hierarchical structure, making it more manageable and easier to navigate.

Code reusability Since the code is created once and can be used by anybody. Thus it provides the feature of reusability of the code and eliminates the chances of duplicating code.

Distribution and installation Packages allow you to distribute and install code as a unit. You can package your code and resources into a distributable package format like `.tar.gz` file. This makes it easier to share your code with others or deploy it on different systems.

3 NumPy (Numerical Python)

3.1 What is NumPy?

NumPy is a powerful open-source scientific package that stands for **Numerical Python**. It uses mathematical and logical operations for handling large datasets through powerful data structures — n-dimensional arrays — that also speed up data processing.

- NumPy is the first step in learning to become a Python data scientist.
- Libraries like Pandas, Matplotlib, and Scikit-learn are built using concepts of NumPy.
- It can be interfaced with other Python packages and provides tools for integrating with languages like C, C++, etc.

3.2 Installation

If you are using basic Python from `https://www.python.org`, NumPy is not included by default. Install it with:

```
pip install numpy
```

3.3 Importing NumPy

```
import numpy                          # complete package
import numpy as np                    # referred as np
from numpy import array               # only arrays
from numpy import array as ary        # only arrays as ary
```

3.4 Arrays

- An array is an ordered collection of values of the same data type (homogeneous) that can be arranged in one or more dimensions.
- Elements are referred to using index numbers starting with 0.
- 1-D array = Vector; 2-D array = Matrix; multi-D = n-dimensional array.
- NumPy arrays are faster alternatives to Python lists for numerical work.

3.5 NumPy Array vs Python List

Aspect	NumPy Array	Python List
Data type	Homogeneous	Heterogeneous
Memory	Less memory	More memory
Access	Through NumPy package	Direct in Python
Math operators	Can be used directly	Must be applied element-wise
Main use	Mathematical operations	Data management

3.6 Creating Arrays

```
import numpy as np

# 1-D array
rollno = np.array([1, 2, 3])
print(rollno)          # [1 2 3]

# Sequential array (multiples of 10 from 10 to 100)
a = np.arange(10, 101, 10)
print(a)

# Random values
a = np.random.random(4)

# 3x4 random integers < 10
a = np.random.randint(10, size=(3, 4))

# All ones
a = np.ones((3, 4))

# All zeros
a = np.zeros((3, 3))

# Fill with a constant
a = np.full((3, 4), 6)

# 2-D from nested lists
a = np.array([[11, 'Arshia', 93], [15, 'Riddhi', 95]])
```

3.7 Mathematical Operators on Arrays

Operators (`+`, `-`, `*`, `/`, `**`, `//`, `%`) work element-wise on NumPy arrays.

```
import numpy as np
marks1 = np.array([7, 8, 5, 4])
marks2 = np.array([5, 6, 4, 7])

print(marks1 + marks2)     # [12 14  9 11]
print(marks1 * 2)          # [14 16 10  8]
print(marks1 ** 3)         # [343 512 125  64]
```

Equivalent functions: `np.add()`, `np.subtract()`, `np.multiply()`, `np.divide()`, `np.remainder()`.

3.8 Important Array Attributes and Methods

Attribute/Method	Description
`type(arr)`	Type of the object
`arr.ndim`	Number of dimensions
`arr.shape`	Shape (length of each dimension)
`arr.size`	Total number of elements
`arr.dtype`	Data type of elements
`arr.max()`	Maximum value
`arr.min()`	Minimum value
`arr.sum()`	Sum of all elements
`arr.max(axis=0/1)`	Column-wise / row-wise maximum
`arr.min(axis=0/1)`	Column-wise / row-wise minimum
`arr.sum(axis=0/1)`	Column-wise / row-wise sum
`np.sort(arr)`	Sort the array

```
import numpy as np
ARR = np.array([[1, 2, 3, 4], [3, 4, 5, 6]])
print(ARR.ndim)         # 2
print(ARR.shape)        # (2, 4)
print(ARR.size)         # 8
print(ARR.max())        # 6
print(ARR.sum(axis=1)) # [10 18]
```

4 Brief Note on Pandas (related library)

Pandas is an open-source library for data manipulation and analysis, built on NumPy. It provides:

- **Series** – 1-dimensional labelled array
- **DataFrame** – 2-dimensional labelled data structure (like a spreadsheet)

Install with: `pip install pandas`

```
import pandas as pd
import numpy as np

# Series from list
cities = ['Delhi', 'Mumbai', 'Chennai', 'Kolkata']
s = pd.Series(cities)

# Series with custom index
friends = pd.Series(['Rohan', 'Susan', 'James'], index=[11, 22, 33])

# Read CSV
df = pd.read_csv("Student_data.csv")
print(df.head(5))
```